👻 GHOST-ENGINE v2

Manual de Arquitetura, Técnicas Anti-Detecção e Roteiro de Replicação para Outros Projetos

PUPPETEER STEALTH | BROWSER POOLING | XHR INTERCEPT | COHERENT PERSONAS | ZERO API KEY | ZERO PROXY PAGO

1. Visão Geral & O Problema Resolvido

O **GhostEngine** é uma arquitetura de extração de dados e automação resiliente desenvolvida para obter dados de Single Page Applications (SPAs) e APIs privadas protegidas por Cloudflare Turnstile, DataDome ou Akamai sem pagar por proxies residenciais ou chaves de API proprietárias.

❌ Paradigma Tradicional (v1)

- Inicia e destrói um Chrome inteiro a cada requisição HTTP.
- **Overhead massivo:** 2 a 4 segundos por busca apenas no handshake do browser.
- Vazamentos de memória (RAM descartada) e picos de 100% de CPU.
- Identidades avulsas facilitam o rastreamento por fingerprinting.

✅ Paradigma GhostEngine v2

- **Processo Chrome Permanente:** Singleton aquecido no boot da API (latência cai para ~800ms).
- **Pool de Abas Reutilizáveis:** Até 6 abas simultâneas que nunca morrem.
- **Intercepção XHR em Tempo Real:** Extração direta da rede antes do DOM.
- **Economia de 70% de Banda:** Bloqueio cirúrgico de recursos visuais.

2. Os 7 Pilares Arquiteturais

PILAR 1 Singleton Browser com Auto-Recuperação

O processo do Chromium é instanciado uma única vez no bootstrap via `ensureBrowser()`. Se o processo do Chrome cair por falta de memória ou sinal externo, o evento `disconnected` limpa o singleton e o pool de abas, recriando a infraestrutura sob demanda na requisição seguinte sem derrubar a API Node.js.

Flag do Chromium	Objetivo Crítico de Defesa e Estabilidade
`headless: 'new'`	Ativa o modo headless moderno (Chromium 112+), eliminando a flag clássica de automação no UA.
`--disable-blink-features=AutomationControlled`	Remove a propriedade nativa `navigator.webdriver` diretamente do núcleo do motor Blink.
`ignoreDefaultArgs: ['--enable-automation']`	Remove a barra amarela de alerta e a flag padrão de controle automatizado do Puppeteer.
`--window-position=-32000,-32000`	Posiciona a janela virtual fora da área de tela visível (zero overhead de desenho gráfico).
`--disable-dev-shm-usage`	Redireciona buffers para disco/RAM geral, prevenindo crashes em Docker com `/dev/shm` de 64MB.

PILAR 2 Firewall de Recursos (Navegação em Modo Texto)

Através de `page.setRequestInterception(true)`, a engine cancela instantaneamente requisições que não carregam dados úteis:

```
const BLOCKED_RESOURCES = new Set([
  'image', 'stylesheet', 'font', 'media', 'manifest', 'texttrack', 'eventsource'
]);
page.on('request', (req) => {
  BLOCKED_RESOURCES.has(req.resourceType()) ? req.abort() : req.continue();
});
```

Impacto Real: Redução de 60% a 70% na banda consumida e queda do tempo de navegação de ~4s para ~1.2s por requisição.

PILAR 3 Sistema de Personas Coerentes (Anti-Stitching Shield)

O Grande Erro do Scraping Comum: Rotacionar aleatoriamente User-Agent em cada requisição dispara alertas imediatos em firewalls (Cloudflare/DataDome). Detectores correlacionam UA com GPU Vendor/Renderer, resolução de tela, device pixel ratio (DPR), fuso horário e ruído do Canvas. Um User-Agent de MacOS com placa de vídeo Direct3D da NVIDIA e fuso de São Paulo é bloqueado no primeiro handshake.

A Solução GhostEngine: O sistema define um conjunto de **Personas Coerentes**. Cada aba aberta é vinculada permanentemente a uma persona desde sua criação e *nunca* altera seus dados. Ela simula com perfeição um computador real específico navegando.

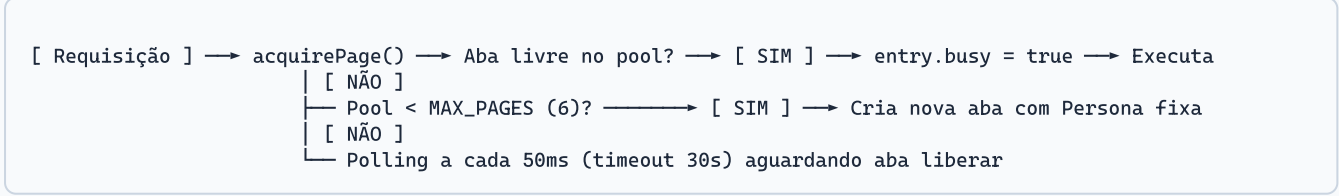
```
const PERSONA_EXEMPLO = {
  ua: 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ... Chrome/124.0.0.0 Safari/537.36',
  viewport: { width: 1920, height: 1080 },
  dpr: 1,
  timezone: 'America/Sao_Paulo',
  glVendor: 'Google Inc. (NVIDIA)',
  glRenderer: 'ANGLE (NVIDIA, NVIDIA GeForce GTX 1660 Direct3D11 vs_5_0 ps_5_0, D3D11)',
  canvasSeed: 0x4a3f
};
```

Injeção de Identidade no DOM (`page.evaluateOnNewDocument`):

Executado *antes* de qualquer script externo da página web ter permissão de execução:

- **Hardware & Navigator:** Força `navigator.webdriver = false`, `deviceMemory = 8` e `hardwareConcurrency = 8`.
- **Objeto Chrome Nativo:** Cria `window.chrome = { runtime: {}, loadTimes: () => {}, csi: () => {}, app: {} }`.
- **Spoofing WebGL:** Intercepta `WebGLRenderingContext.getParameter` retornando `37445 (VENDOR)` e `37446 (RENDER ER)` fiéis à persona.
- **Ruído Determinístico de Canvas:** Aplica variação de ±1 em ~0.1% dos pixels usando um LCG com a `canvasSeed` da persona. Isso gera um fingerprint único por aba, porém 100% consistente ao longo de toda a sessão da mesma aba.
- **Fuso Horário Coerente:** Proxy no `Intl.DateTimeFormat` forçando o fuso da persona.

PILAR 4 Pool de Abas (Acquisition & Release)



Quando a busca termina, `releasePage(entry)` apenas define `entry.busy = false`. A aba **não é fechada**; ela permanece com a sessão quente na memória pronta para a próxima demanda.

PILAR 5 Intercepção de Rede XHR/Fetch em Tempo Real

Em vez de esperar o DOM terminar de renderizar e buscar classes CSS frágeis, o motor intercepta o pacote JSON diretamente no fluxo de tráfego HTTP:

```
let interceptedData = null;
const onResponse = async (res) => {
  if (res.url().startsWith('https://api.alvo.com') && res.status() === 200 &&
      (res.headers()['content-type'] ?? '').includes('json')) {
    interceptedData = await res.json();
  }
};
page.on('response', onResponse);
await page.goto(targetUrl, { waitUntil: 'domcontentloaded', timeout: 25000 });
page.off('response', onResponse);
```

Com `waitUntil: 'domcontentloaded'`, a navegação termina no primeiro segundo. Se o XHR capturar os dados, o processamento encerra imediatamente. Se houver falha de rede na captura, entra o fallback: `document.body.innerText`.

PILAR 6 Timing Humano (Box-Muller) & Retries

Intervalos de tempo lineares (ex: `Math.random() * 500`) entregam um padrão sintético a sistemas anti-bot.

O GhostEngine utiliza a **Transformada de Box-Muller** para criar uma distribuição Log-Normal de atraso humano com pico em 250ms (80ms a 900ms).

Retries com Backoff Exponencial: Tentativa 1 (1s de delay), Tentativa 2 (2s de delay), Tentativa 3 (4s). Se falhar, libera a aba e propaga o erro de forma limpa.

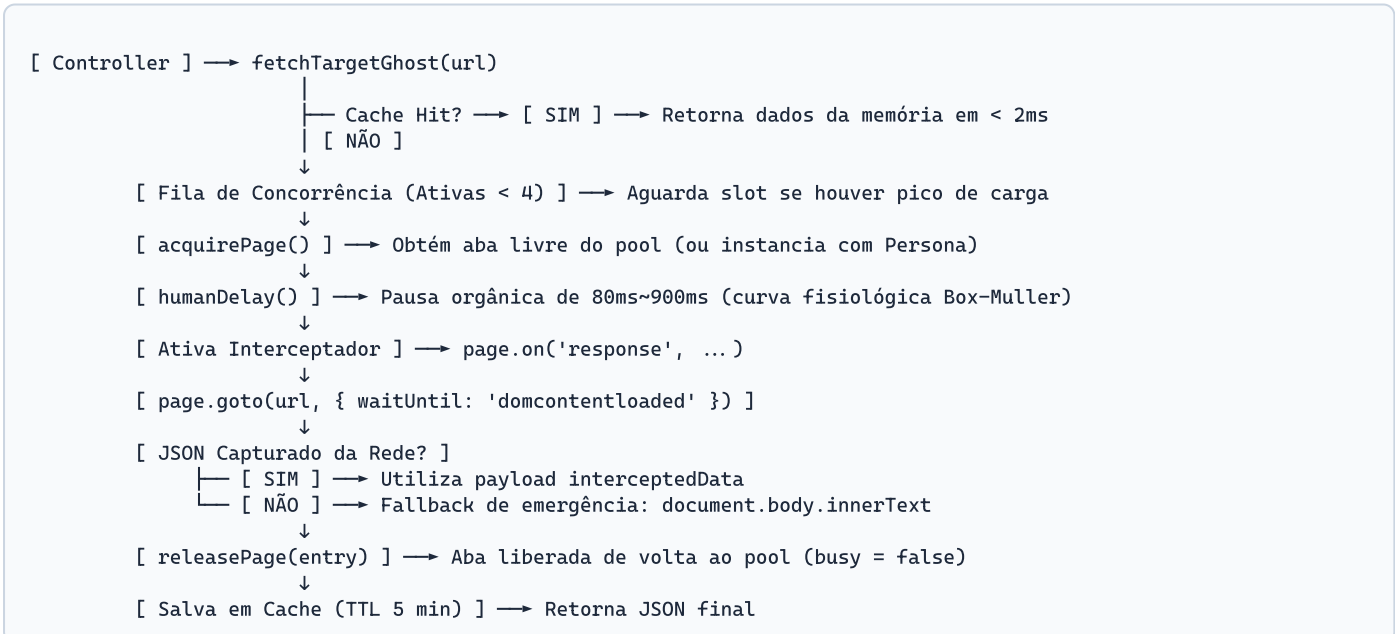
PILAR 7 Semáforo de Concorrência & Cache

Fila de Concorrência (MAX_CONCURRENT = 4): Permite no máximo 4 extrações paralelas. Novas requisições entram em uma fila FIFO em memória e são despachadas automaticamente assim que um slot é desocupado.

Cache em Memória (TTL 5 min): Evita requisições repetidas para a mesma entidade. Respostas indexadas por URL em um `Map` com timestamp de expiração (latência < 2ms).

Pre-warm Automático: O browser é inicializado na importação do módulo, eliminando a latência da primeira requisição.

3. Fluxo Completo de Uma Chamada



4. Roteiro Prático de Replicação para um Novo Projeto

Passo 1: Dependências Essenciais

```
npm install puppeteer-extra puppeteer-extra-plugin-stealth puppeteer
```

Passo 2: Configuração e Exportações do Módulo

Função / Objeto	Assinatura	Função no Projeto
`fetchTrackerGhost`	`(url: string) ⇒ Promise<any>`	Ponto de entrada obrigatório. Gerencia fila de concorrência, cache, pool e retries.
`getCached`	`(key: string) ⇒ any | null`	Leitura manual da memória sem disparar tráfego de rede.
`flushCache`	`() ⇒ void`	Limpeza completa da memória (útil para testes unitários ou endpoints de reload).

Passo 3: Adaptação das Regras para o Novo Alvo

1. **Alvo da Intercepção:** Altere a linha `res.url().startsWith( ... )` para o padrão da API interna do seu alvo.
2. **Cabeçalhos Específicos:** Caso o alvo exija headers particulares (ex: `Accept-Language` em inglês ou `sec-ch-ua` atualizado), ajuste em `createPage()`.
3. **Dimensionamento de Recursos:** Para VPS pequenas (1GB RAM), configure `MAX_PAGES = 3` e `MAX_CONCURRENT = 2`. Para servidores dedicados (4GB+ RAM), pode-se elevar para `MAX_PAGES = 10`.

Passo 4: Cuidados em Produção e Ambientes Docker

- **Dependências de Sistema:** No Linux/Docker, instale os pacotes: `libnss3 libatk1.0-0 libatk-bridge2.0-0 libcups 2 libdrm2 libxkbcommon0 libxcomposite1 libxdamage1 libxfixes3 libxrandr2 libgbm1 libasound2`.
- **Flags de Container:** As flags `--no-sandbox` e `--disable-dev-shm-usage` já inclusas no GhostEngine são obrigatórias em Docker para prevenir crashes instantâneos.
- **Bloqueio de IP de Datacenter:** Se a API rodar em AWS/DigitalOcean e o alvo bloquear o bloco de IP do datacenter, passe um proxy residencial nas flags do Chromium: `--proxy-server=http://usuario:senha@proxy-host:porta`.

Resumo Executivo para a Equipe:

O GhostEngine alcança a combinação ideal entre **eficiência operacional** (pool de instâncias ativas + bloqueio de recursos + bypass de DOM) e **máxima proteção contra detecção** (personas coerentes + spoofing profundo + curvas estatísticas humanas). É um padrão de design maduro, estável e pronto para ser adotado em qualquer microsserviço.